

《Actions Speak Louder than Words》 中文翻译整理版

用于生成式推荐的万亿参数序列转导器

Actions Speak Louder than Words: Trillion-Parameter Sequential
Transducers for Generative Recommendations

作者: Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao,
Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, Yinghai Lu, Yu Shi

来源: ICML 2024 / arXiv:2402.17152v3

说明: 本文档依据用户上传的论文制作中文翻译整理版。正文、附录主要文字、公式与算法说明均已翻译为中文; 参考文献未逐条翻译, 仍建议结合原论文使用。图表中的数值、符号、变量名与编号尽量沿用原文记法, 以便对照阅读。

生成日期: 2026 年 4 月 2 日

目录

1 引言 (Introduction)	3
2 将推荐问题表述为序列转导任务：从 DLRM 到 GR	4
2.1 统一 DLRM 中的异构特征空间	4
2.2 将排序与检索重构为序列转导任务	5
2.3 生成式训练	5
3 面向生成式推荐的高性能自注意力编码器	6
3.1 逐点聚合注意力 (Pointwise Aggregated Attention)	6
3.2 利用并进一步增加稀疏性	7
3.3 降低激活内存占用	7
3.4 通过代价摊销扩展推理能力	8
4 实验 (Experiments)	8
4.1 验证 HSTU 编码器的归纳假设	8
4.1.1 传统序列推荐设置	8
4.1.2 工业级流式设置	8
4.2 编码器效率	9
4.3 工业级流式环境中：GR 与 DLRM 的端到端比较	9
4.4 推荐系统中的扩展律 (Scaling Law)	10
5 相关工作 (Related Work)	10
6 结论 (Conclusions)	10
A 附录 A：符号说明 (Notations)	11
B 附录 B：生成式推荐器的背景与问题表述	13
B.1 学术界与工业界中的序列推荐背景	13
B.2 GR 如何把排序与检索表述为序列转导任务	14
C 附录 C：合成数据实验 (Synthetic Data)	14

D	附录 D：传统序列推荐设置下的进一步评估	15
E	附录 E：传统 DLRM 基线说明	15
F	附录 F：Stochastic Length	16
F.1	子序列选择 (Subsequence Selection)	16
F.2	Stochastic Length 对序列稀疏性的影响	16
F.3	与序列长度外推技术的比较	16
G	附录 G：稀疏分组 GEMM 与融合相对注意力偏置	17
H	附录 H：M-FALCON 详解	17

摘要 (Abstract)

大规模推荐系统的一个核心特征，是高度依赖高基数、异构化特征，并且需要每天处理数百亿次用户行为。尽管工业界的大多数深度学习推荐模型 (Deep Learning Recommendation Models, DLRMs) 已经使用海量数据和成千上万种特征进行训练，但其性能并不能随计算量有效扩展。受 Transformer 在语言与视觉领域成功经验的启发，作者重新审视了现代推荐系统中的若干基础设计选择。

本文将推荐问题重新表述为生成式建模框架下的序列转导任务 (作者称之为 “Generative Recommenders”，简称 GR)，并提出一种面向高基数、非平稳流式推荐数据的新架构 HSTU。实验表明，HSTU 在合成数据和公开数据集上相较基线最高可带来 65.8% 的 NDCG 提升；在长度为 8192 的序列上，相比基于 FlashAttention2 的 Transformer，HSTU 快 5.3× 到 15.2×。基于 HSTU 的生成式推荐模型在参数规模达到 1.5 万亿时，在线 A/B 测试指标提升了 12.4%，并已部署在拥有数十亿用户的大型互联网平台多个业务场景中。

更重要的是，生成式推荐模型的质量在经验上遵循关于训练算力的幂律扩展关系，而且这一现象跨越了三个数量级，一直延伸到 GPT-3 / LLaMa-2 级别的训练规模。这意味着后续模型研发所需碳足迹有望下降，也为推荐领域中的首批基础模型铺平了道路。

1 引言 (Introduction)

推荐系统是在线内容平台与电子商务系统中的关键组成部分，每天都在影响着数十亿用户的个性化体验。过去十余年中，工业推荐系统的主流方法基本都建立在 DLRM 范式之上。DLRM 的典型特征，是同时使用大量异构特征：一类是数值型特征，例如计数器、比例值、连续统计量；另一类是类别型特征，例如创作者 ID、用户 ID、内容 ID 等。由于新内容与新商品以极高频率持续产生，整个特征空间通常具有极端高基数，往往达到十亿级规模。为了利用成千上万种此类特征，DLRM 往往需要组合多种神经网络结构，分别承担特征融合、中间表示变换以及最终输出构造等职责。

然而，尽管这类系统拥有大量人工设计特征，并在海量数据上训练，工业界大多数 DLRM 的质量却并不能随着计算量继续有效提升。这一局限非常突出，但长期以来并没有被真正解决。

受 Transformer 在语言与视觉中的成功启发，作者重新审视了现代推荐系统的基本设计。作者指出，在十亿用户规模下，若要采用新的问题表述方式，至少需要跨越三项关键挑战。其一，推荐系统中的特征通常缺乏类似自然语言那样的显式结构。虽然序列化建模在小规模环境下已有探索，但工业级 DLRM 仍高度依赖高基数 ID、交叉特征、计数器和比例等异构输入。其二，推荐系统面对的是持续变化的十亿级词表，而非语言模型中相对稳定的十万级静态词表；这不仅增加训练难度，也使目标感知 (target-aware) 推理在需要同时考虑成千上万个候选项时成本高昂。其三，计算开销本身就是大规模序列模型在推荐场景落地的主要瓶颈。GPT-3 在数千块 GPU 上用 1-2 个月处理了 300B 个 token，但与之

相比，大型互联网平台每天要处理的用户行为 token 数量其实还要大上若干个数量级。

本文的核心出发点，是将用户行为视为生成式建模中的一种新模式。作者认为：(1) 在合适的新特征空间中，工业推荐中的核心检索与排序任务可以统一表述为生成式建模问题；(2) 一旦完成这种重构，就可以系统性利用特征、训练与推理中的冗余来提升效率。基于这一表述，作者得以部署比此前最先进系统复杂三个数量级的模型，同时在线核心指标提升了 12.4%。

本文贡献主要包括三个方面。第一，作者提出 **Generative Recommenders (GR)** 这一全新范式，用序列化方式统一 DLRM 中的异构特征空间；当序列长度趋向无穷时，这种表示能够逼近完整 DLRM 特征空间。在此基础上，排序与检索都可被重写为纯粹的序列转导任务，而且模型可以按生成式方式进行训练，从而在相同计算预算下处理更多训练数据。第二，作者针对训练与推理阶段的计算瓶颈提出 HSTU 架构。HSTU 针对大规模、非平稳词表修改了注意力机制，并利用推荐数据的统计特征，在长序列设置下相较 FlashAttention2-Transformer 取得了显著提速。进一步地，作者还设计了 M-FALCON 算法，通过微批处理 (micro-batching) 充分摊销计算成本，在与传统 DLRM 相同的推理预算下，可以服务复杂度高出 285 倍的 GR 模型，同时仍获得 $1.50\times$ – $2.99\times$ 的速度提升。第三，作者在合成数据、公开数据集以及真实工业部署场景中验证了整套方法。作者认为，这是首次在大规模工业推荐环境中证明，类似 HSTU 这种纯序列转导式架构在生成式推荐设置下能够显著超过 DLRM，也首次展示了推荐系统中同样存在类似大语言模型中的 scaling law 现象。

2 将推荐问题表述为序列转导任务：从 DLRM 到 GR

2.1 统一 DLRM 中的异构特征空间

现代 DLRM 通常同时接收大量类别型 (sparse) 和数值型 (dense) 特征。在 GR 中，作者将这些特征统一整理并编码到一条时间序列中。

对于 **类别型特征**，典型例子包括：用户点赞过的内容、用户关注的某类创作者、用户语言、加入的社区、发起请求的城市等。作者的做法是，**先选取最长的一条时间序列作为主时间序列，通常是与用户交互内容相关的序列**；其余类别型特征往往变化更慢，例如人口属性、关注创作者等，于是先对这些 **序列做压缩，仅保留连续区段中的最早条目，再将其并入主时间序列**。由于这些辅助序列变化很慢，这种合并不会显著增加整体序列长度。

对于 **数值型特征**，典型例子包括加权计数器、衰减计数器、比例等。例如，一个特征可以表示用户过去在某一话题上的点击率 (CTR)。与类别型特征不同，**数值型特征更新频率极高**，甚至可能在每一次用户-物品交互后都发生变化，因此从计算和存储角度看，不可能将它们完整地逐项序列化。作者的重要观察是：这些数值特征所依赖的类别基础，例如话题、位置、实体等，实际上已经在 GR 中通过序列形式表达出来了。因此，**只要序列转导架构足够有表达能力，并结合目标感知建模方式，就有可能随着序列长度和计算量增加，通过模型本身去隐式捕获这些数值特征，从而在 GR 中去除显式 dense 特征。**

2.2 将排序与检索重构为序列转导任务

设按时间顺序观测到一个长度为 n 的 token 序列 $x_0, x_1, \dots, x_{n-1}$, 对应观测时刻为 $t_0, t_1, \dots, t_{n-1}$ 。所谓序列转导任务, 就是把该输入序列映射到输出 token 序列 $y_0, y_1, \dots, y_{n-1}$, 其中 $y_i = \emptyset$ 表示该位置无定义输出。作者用 $\Phi_i \in X_c$ 表示系统向用户提供的内容 (例如图像或视频), 由于新内容持续产生, 内容集合 X_c 及整体词表 X 都是非平稳的。用户会对 Φ_i 做出动作 a_i , 如点赞、跳过、看完并分享等。记用户交互过的内容总数为 n_c 。

在这一框架下, 工业推荐中的检索与排序可以统一写成因自回归条件下的序列转导任务。对于检索, 系统学习分布 $p(\Phi_{i+1} | u_i)$, 其中 u_i 是时刻 i 的用户表示, 目标通常是选出使该分布最大化的候选内容。它与标准语言建模不同的地方在于: 其监督信号不一定是 Φ_{i+1} 本身, 因为用户也可能对下一条内容作出负反馈; 同时, 当 x_{i+1} 对应的是人口属性等非交互类特征时, y_i 可能并没有定义。

对于排序, 工业推荐往往需要目标感知建模, 也就是说, 候选目标项 Φ_{i+1} 与历史特征的交互必须尽早发生。而在标准自回归模型中, 目标项往往只在编码器输出后通过 softmax 等操作晚期进入模型, 这并不适合推荐排序。作者因此采用“内容-动作交织”的输入方式, 将项目与用户动作交替排列, 从而把排序问题写成

$$p(a_{i+1} | \Phi_0, a_0, \Phi_1, a_1, \dots, \Phi_{i+1}),$$

并在实践中对 Φ_{i+1} 位置的输出再接一个小型神经网络, 用于生成多任务预测。这样便能够在一次前向传播中, 对所有 n_c 个交互位置同时施加目标感知交叉注意力。

2.3 生成式训练

工业推荐系统通常采用流式训练方式, 即每当新的训练样本到达, 就按时间顺序处理。在这种设置下, 若使用 Transformer 等自注意力序列模型, 整体计算量会随用户序列长度快速膨胀。若第 i 个用户序列长度为 n_i , 则训练代价大致为

$$\sum_i n_i(n_i^2 d + n_i d_{\text{ff}} d),$$

其中前一项来自自注意力, 后一项来自逐点 MLP。当取 $N = \max_i n_i$ 时, 复杂度可化为 $O(N^3 d + N^2 d^2)$, 这在推荐场景中成本过高。

为了解决长序列上的可扩展训练问题, 作者将传统的 impression 级训练改为生成式训练, 使计算复杂度额外下降一个 $O(N)$ 因子。直观上看, 编码器的计算成本能够在多个目标之间被摊销。更具体地, 若以概率 $s_u(n_i)$ 采样第 i 个用户, 则总体训练成本变为

$$\sum_i s_u(n_i) n_i(n_i^2 d + n_i d^2),$$

当取 $s_u(n_i) = 1/n_i$ 时, 这一代价降为 $O(N^2 d + N d^2)$ 。在工业系统中, 实现这一采样的一种自

然方式，就是在用户一次请求或会话结束时再统一发射训练样本，这对应着 $\hat{s}_u(n_i) \propto 1/n_i$ 。

3 面向生成式推荐的高性能自注意力编码器

为了让 GR 在工业大规模推荐系统中可扩展，作者提出新的编码器结构 **Hierarchical Sequential Transduction Unit (HSTU)**。HSTU 由若干完全相同的层堆叠而成，各层之间通过残差连接相连。每一层包含三个子层：逐点投影 (Pointwise Projection)、空间聚合 (Spatial Aggregation) 和逐点变换 (Pointwise Transformation)：

$$U(X), V(X), Q(X), K(X) = \text{Split}(\phi_1(f_1(X))), \quad (1)$$

$$A(X)V(X) = \phi_2(Q(X)K(X)^T + r_{ab}^{p,t})V(X), \quad (2)$$

$$Y(X) = f_2(\text{Norm}(A(X)V(X)) \odot U(X)). \quad (3)$$

其中， $f_i(X)$ 表示 MLP；为了降低计算复杂度，作者在 f_1 与 f_2 中都只使用单层线性变换，即 $f_i(X) = W_i(X) + b_i$ ，并借助融合核 (fused kernel) 对查询、键、值以及门控向量的计算进行批处理。 ϕ_1 和 ϕ_2 是非线性函数，文中均采用 SiLU；Norm 表示层归一化； $r_{ab}^{p,t}$ 表示同时编码相对位置与相对时间信息的注意力偏置。

作者指出，HSTU 使 DLRM 中大量异构模块有可能被单一模块化结构取代。DLRM 可以粗略分为三个阶段：特征提取、特征交互以及表示变换。特征提取阶段主要取得类别特征的池化表示，其更复杂形式往往可概括为成对注意力与目标感知池化，这在 HSTU 中可通过注意力层统一表达。特征交互是 DLRM 最关键的部分，常用方法包括因子分解机及其神经变体、更高阶交互结构等；在 HSTU 中，这些交互通过 $\text{Norm}(A(X)V(X)) \odot U(X)$ 的方式完成。至于表示变换，工业模型常借助 MoE 等结构实现条件计算，而 HSTU 中的逐元素乘积在适当归一化后，也可看作一种近似门控机制。

3.1 逐点聚合注意力 (Pointwise Aggregated Attention)

HSTU 采用一种新的逐点聚合（而非序列整体 softmax 归一化）注意力机制。作者提出这一设计主要基于两点考虑。第一，与目标相关的历史事件数量本身就是刻画用户偏好强度的重要信号，而在 softmax 归一化后，这种“数量强度”信息会被削弱。这一点对推荐尤为重要，因为系统不仅要判断物品间的相对排序，还要预测参与强度，例如用户在某类内容上可能投入的时长。第二，虽然 softmax 天然具有一定抗噪性，但面对非平稳词表的流式场景时，它并不一定是最合适的选择。

作者在合成数据上验证了这一点。该合成数据依据 Dirichlet Process 构造，以模拟动态词表下的流式用户行为序列。实验结果显示，在这种环境中，softmax 注意力与逐点注意力之间的差距可以高达 44.7%。作者因此认为，逐点聚合式注意力更适合大规模推荐场景。

3.2 利用并进一步增加稀疏性

推荐系统中的用户历史长度通常服从偏斜分布,尤其在长序列设置下,输入天然呈现稀疏特征。作者为此开发了高效注意力核:其思路与 Memory-efficient Attention 和 FlashAttention 类似,但能够完全以“不规则张量”(raggified attention)的形式执行注意力计算,将问题重写为一组形状不同的分组 GEMM 运算。这样,自注意力在 HSTU 中更偏向内存访问受限,其内存访问复杂度可写为

$$\Theta\left(\sum_i n_i^2 d_{qk} R^{-1}\right),$$

其中 n_i 是样本 i 的序列长度, d_{qk} 是注意力维度, R 是寄存器大小。仅凭这一点,作者在实验中就获得了 $2\times$ 到 $5\times$ 的吞吐提升。

在此基础上,作者进一步通过 **Stochastic Length (SL)** 从算法上增加稀疏性。推荐中的用户行为在时间上具有多尺度重复性,这意味着可以在不明显伤害模型质量的前提下,主动对长历史进行稀疏抽样。设用户 j 的完整交互历史为 $(x_i)_{i=0}^{n_{c,j}}$,从中抽取长度为 L 的子序列 $(x_{i_k})_{k=0}^L$ 。SL 的选择规则是:若 $n_{c,j} \leq N_c^{\alpha/2}$,则保留全序列;若 $n_{c,j} > N_c^{\alpha/2}$,则以高概率保留长度为 $N_c^{\alpha/2}$ 的子序列,以低概率仍保留完整序列。由此,注意力相关复杂度可降为 $O(N^\alpha d)$,其中 $\alpha \in (1, 2]$ 。作者给出的统计结果表明,在 30 天用户历史配置下,只要合理设置 α ,即使稀疏度达到 64%–84%,模型主任务上的 NE 几乎不劣化。

3.3 降低激活内存占用

在推荐场景中,大 batch size 对训练吞吐与模型质量都非常重要,因此激活内存往往成为主要的扩展瓶颈。相较于 Transformer, HSTU 在结构上更加简化,也更彻底地将多步计算融合到单个算子中,所以激活内存显著更低。

具体来说,Transformer 在注意力之外通常还包含较大的前馈网络和 dropout,中间态数量较多;而 HSTU 把线性层数量从六层减少到两层,并将 $\phi_1(f_1(\cdot))$ 、层归一化、可选 dropout 以及输出 MLP 等过程尽可能融合。作者估算,在 bfloat16 下, HSTU 每层的激活状态约为 $14d$,而 Transformer 可达 $33d$ 。这使 HSTU 能在同等显存下堆叠超过 2 倍深度的网络。

另一方面,推荐系统中十亿级原子 ID 词表本身也占用巨量内存。例如,当词表规模为 10B、嵌入维度为 512、优化器为 Adam 且参数用 fp32 存储时,仅嵌入与优化器状态就可能需要约 60TB 内存。为缓解这一压力,作者采用 rowwise AdamW,并将优化器状态放在 DRAM 而非 HBM 上,从而把每个 float 在 HBM 上的占用从 12 字节降到 2 字节。

3.4 通过代价摊销扩展推理能力

作者最后解决的问题，是排序阶段需要同时处理大量候选项。对于检索，编码器成本本就被摊销，而且已有大量针对 MIPS 和非 MIPS 场景的高效算法；真正困难的是排序，因为工业排序通常需要同时评估数千到上万个候选项。

为此，作者提出 **M-FALCON** (Microbatched-Fast Attention Leveraging Cacheable Operations) 算法。设输入序列长度为 n 、候选数为 m 。M-FALCON 在一次前向传播中并行处理 b_m 个候选，通过修改注意力 mask 和相对偏置，使得这 b_m 个候选共享同一套历史上下文，但彼此之间不能互相注意。这样，原本复杂度为 $O(b_m n^2 d)$ 的目标感知交叉注意力，便可化为 $O((n + b_m)^2 d)$ ；当 b_m 相对 n 为小常数时，这一代价近似为 $O(n^2 d)$ 。如果总候选数 m 更大，还可以把候选划分成 $\lceil m/b_m \rceil$ 个微批，并叠加 KV cache，在微批之间乃至不同请求之间共享顺序编码部分的计算。

总体而言，M-FALCON 使模型复杂度能够随着候选数线性扩展。作者在典型排序配置下证明：即便使用比传统 DLRM 排序模型复杂 285 倍的目标感知交叉注意力模型，依然可以在固定推理预算下获得约 $1.5\times-3\times$ 的吞吐提升。

4 实验 (Experiments)

4.1 验证 HSTU 编码器的归纳假设

4.1.1 传统序列推荐设置

作者首先在 MovieLens 与 Amazon Reviews 两个常用公开数据集上评估 HSTU，采用文献中流行的传统序列推荐设置，包括 full shuffle 和 multi-epoch 训练。基线使用的是 SASRec，这是一种广泛采用的 Transformer 序列推荐实现。评价指标为全语料上的 Hit Rate@K 与 NDCG@K。

结果表明，在使用与 SASRec 相同层数、头数等配置时，HSTU 就已经显著超过基线；而当进一步扩大层数和头数后，HSTU-large 还能继续提升。作者强调，这类“多轮打乱、多 epoch”的评测方式与工业流式训练环境差异较大，因为工业场景一般并不具备对完整历史反复洗牌训练的条件。

4.1.2 工业级流式设置

接着，作者在更接近真实部署的大规模流式数据集上，对 HSTU、删减版 HSTU 以及 Transformer 进行比较。排序任务使用 Normalized Entropy (NE) 评价，检索任务报告对数困惑度。排序场景中，作者同时给出主要参与任务 (E-Task) 与主要消费任务 (C-Task) 的结果。文中指出，在其业务环境里，NE 降低 0.001 通常就意味着面向数十亿用户的线上核心指标约可提升 0.5%，因此已经是非常显著的改进。

结果显示，HSTU 明显优于标准 Transformer，尤其在排序任务上优势更大；作者认为这主要来自逐点注意力和更好的相对偏置设计。删减版 HSTU 与完整 HSTU 之间的差距也说明了各设计元素均有贡献。作者还观察到，基于 softmax 的 HSTU 与 Transformer 需要更小学习率才能勉强稳定训练，即便采用 pre-norm 残差结构，标准 Transformer 在排序任务中仍频繁出现 loss explosion。整体而言，在这一较小规模但更贴近工业的设置中，HSTU 以约 $1.5\times-2\times$ 更快的训练/推理速度和约一半的 HBM 占用，实现了更优质量。

4.2 编码器效率

关于 **Stochastic Length**，作者在不同最大序列长度设置下分析其对主任务指标的影响。以 $\alpha = 1.6$ 、最大长度 4096 为例，经过 SL 采样后，序列通常会缩短到约 776，仅保留不到 20% 的 token。但即便稀疏率达到 64%–84%，主任务的 NE 退化仍不到 0.002，即不到 0.2%。这说明在合适的参数范围内，SL 可以显著降低训练成本，而不会实质伤害模型质量。

关于 **编码器效率**，作者将 HSTU 与使用 FlashAttention2 的 Transformer 在训练与推理场景中直接比较。实验覆盖 1024 到 8192 的多种序列长度，并在训练时启用 SL。结果表明，在相同配置下，HSTU 在训练与推理中分别可达到最高 $15.2\times$ 与 $5.6\times$ 的效率优势。再加上激活内存更低，HSTU 在给定硬件条件下还能构建比 Transformer 深得多的网络。

4.3 工业级流式环境中：GR 与 DLRM 的端到端比较

作者最后把 GR 与工业级最强 DLRM 基线进行端到端比较。需要说明的是，GR 的实现代表了一种典型生产配置，而对照 DLRM 则是多个团队多年工程迭代的结果。对于检索，由于实际系统通常有多个召回源，作者既报告“新增一个 GR 召回源”的线上结果，也报告“用 GR 替换现有主召回源”的结果。

结果表明，无论在线下还是线上，GR 都明显优于 DLRM。在线 A/B 测试中，GR 带来了最高 12.4% 的指标提升。一个关键原因在于：GR 只依赖原始类别型交互特征，而 DLRM 往往还需要大量基于原始信号手工构造的辅助特征。如果把与 GR 相同的特征子集拿来训练 DLRM，其性能会大幅下降，这说明 GR 的统一特征空间和序列化架构确实能够有效“吸收”原本由人工特征完成的信息表达。

作者还把 GR 与传统“只看交互历史”的序列推荐设定进行比较，发现后者在检索和排序上都明显更差；同时，若仅用内容特征、不使用高基数用户行为，性能又会大幅落后。这进一步说明：用户行为序列本身才是推荐建模中最重要、但此前被低估的一类生成式模态。

在推理效率方面，虽然 GR 模型在 FLOPs 上比生产 DLRM 高出 285 倍，但在对 1024 / 16384 个候选进行打分时，借助 HSTU 与 M-FALCON，系统仍实现了 $1.50\times$ / $2.99\times$ 更高的 QPS。这是本文最有工业冲击力的结果之一。

4.4 推荐系统中的扩展律 (Scaling Law)

工业界常见经验是：DLRM 当参数量和算力达到一定规模后，模型质量会趋于饱和。作者希望进一步理解这一现象，因此比较了 DLRM 与 GR 随算力增长的扩展能力。

在排序设置中，作者尝试使用 Transformer、DHEN 以及带残差的 DCN 变体来扩展 DLRM；在检索设置中，则扩大隐藏层宽度、嵌入维度和层数。对于 GR，则通过调整 HSTU 的层数、序列长度、嵌入维度、头数等超参数进行扩展，并在检索任务中增加负样本数量。

实验结果表明，在低算力区间，DLRM 可能因为拥有更多人工特征而暂时优于 GR；但随着算力增加，DLRM 很快进入平台期，而 GR 则展现出显著更强的扩展性。作者不仅在 FLOPs 维度上观察到这一点，也在嵌入参数量与非嵌入参数量上看到相同趋势。最终，GR 成功扩展到 1.5 万亿参数级别，而 DLRM 大约在 2000 亿参数附近便趋于饱和。

更进一步，检索的 Hit Rate@100、Hit Rate@500 以及排序的 NE，都经验性地遵循关于训练算力的幂律关系，而且这一规律跨越三个数量级，一直延伸到与 GPT-3、LLaMa-2 相当的总训练算力。作者同时指出，与语言建模相比，序列长度在 GR 中的重要性更高，因此必须与其他模型参数联合扩展。这也是本文最重要的结论之一：大语言模型中的 scaling law 也可能同样适用于大规模推荐系统。

5 相关工作 (Related Work)

早期序列推荐工作通常把用户行为压缩为由物品组成的一条单一同质序列。工业界对序列方法的采用，则多以成对注意力或将顺序编码器作为 DLRM 的一个子模块的方式出现。为提高效率，也有工作探索多阶段注意力结构来替代标准自注意力。把 ID 表示为 token 序列并用于检索的生成式方法也曾出现过。

高效注意力是另一个与本文密切相关的研究方向。由于自注意力存在 $O(n^2)$ 的复杂度，研究者提出了因子化注意力、低秩近似、线性注意力以及硬件友好的各种变体。HSTU 的逐元素门控设计在灵感上与 FLASH 等工作有关，而 Stochastic Length 则可视作把“随机性”引入长度维，而不是深度维。

近年来，大语言模型与推荐结合也成为热点。已有研究尝试把推荐任务表述为上下文学习、指令微调或迁移学习问题；也有工作利用 LLM 中蕴含的世界知识改善零样本或小样本推荐。然而作者指出，这些工作大多集中在低数据规模环境，在大规模设置下，尚未稳定超过基于协同过滤的强基线。

6 结论 (Conclusions)

本文提出了 Generative Recommenders (GR) 这一新范式，把排序与检索统一建模为序列转导任务，并以生成式方式进行训练。支撑这一范式的是新型 HSTU 编码器：在长度为 8192 的序列上，它比最先进的 Transformer 快 $5.3\times$ – $15.2\times$ ，并可与 M-FALCON 等新型

训练/推理算法协同工作。借助这一整套设计，作者得以在更低推理成本下部署复杂度高出 285 倍的模型，并在真实生产环境中带来 12.4% 的指标提升。

作者认为，用户行为是生成式建模中长期被忽视的一种模态，这也是标题 “Actions Speak Louder than Words” 所要表达的核心观点。更进一步，由于本文极大简化了推荐系统中的特征体系，并建立了统一的序列化特征空间，它为推荐、搜索和广告领域的基础模型打开了可能性；而 GR 的完全序列化设置，也为把推荐进一步推向端到端生成式系统提供了基础。

影响说明 (Impact Statements)

作者认为，本文工作具有广泛的正向影响。首先，若推荐、搜索和广告系统能够减少对大量异构人工特征的依赖，那么这些系统有望在提升用户体验的同时，变得更加隐私友好。其次，若系统能够以完全序列化的方式，把用户长期结果与短期推荐决策联系起来，就有机会降低诱导点击内容、标题党和虚假新闻的传播，从而更好地让平台激励与用户长期价值对齐。最后，若基础模型与 scaling law 真能在推荐中成立，那么后续模型研发也可能以更低碳足迹实现更高性能。

致谢 (Acknowledgements)

作者指出，这项工作是数百人共同努力的结果。原文在致谢部分按字母顺序详细列出了大量参与者名单，并感谢了产品支持、项目管理支持以及管理层与研究同事的讨论和领导。由于该部分主要为人名清单，中文翻译版不逐条展开，建议读者直接对照原文查看。

A 附录 A：符号说明 (Notations)

原文表 8 和表 9 总结了全文主要记号。为方便中文阅读，这里给出对应解释的整理版。

符号	中文说明
$\Psi_k(t_j)$	特征日志系统在时刻 t_j 发出的第 k 个训练样本。典型 DLRM 中，系统会把用户对内容 Φ_i 的动作 a_i 与当时用于排序 Φ_i 的特征拼接起来，形成训练样本。文中指出，GR 与 DLRM 的训练样本数量不同，GR 通常少 1-2 个数量级。
$n_c, n_{c,i}$	用户（或样本 i ）交互过的内容数量。
$\Phi_0, \dots, \Phi_{n_c-1}$	在推荐场景中，用户交互过的内容序列。
$a_0, \dots, a_{n_c-1}$	与各个 Φ_i 对应的动作序列。若预测事件均为二值，则一个动作也可视为对点赞、分享、评论、图片查看、视频开始、视频完成、隐藏等原子事件的 multi-hot 向量。

符号	中文说明
E, F	图 2 中 DLRM 的类别特征，代表由历史交互 (Φ_i, a_i, t_i) 在不同时间点经过特征抽取得到的派生表示。“merge & sequentialize”表示从这些派生特征逆向恢复原始交互序列。
G, H	图 2 中与用户-内容交互无关的类别特征，例如人口属性、关注创作者等；这些特征会被合并进主时间序列。
n, n_i	序列转导任务中的 token 数。虽然通常 $O(n) = O(n_c)$ ，但即便不考虑额外类别特征， n 与 n_c 也未必严格相等。
$x_0, \dots, x_{n-1}$	序列转导任务的输入 token 序列。
$y_0, \dots, y_{n-1}$	序列转导任务的输出 token 序列。
$t_0, \dots, t_{n-1}$	各输入 token 被观察到的时间戳。
X, X_c	全体输入/输出 token 词表，以及其中的内容子集。
N, N_c	分别表示 $\max_i n_i$ 与 $\max_i n_{c,i}$ 。
u_t	时刻 t 的用户表示。
$s_u(n_i), \hat{s}_u(n_i)$	生成式训练中对用户 i 的采样率。
d	模型维度（嵌入维度）。
d_{qk}	HSTU / Transformer 中的注意力维度，对应 $Q(X)$ 与 $K(X)$ 。
d_v	HSTU 中的 value 维度；Transformer 中通常有 $d_{qk} = d_v$ 。
d_{ff}	Transformer 前馈层隐藏维度。HSTU 不使用传统前馈层。
h	注意力头数。
l	HSTU 层数；对 Transformer 而言，一层通常包含注意力与前馈两个部分。
X (HSTU 层输入)	输入到某个 HSTU 层的张量。在未 batch 化时，若序列长度为 N ，则 $X \in \mathbb{R}^{N \times d}$ 。
$Q(X), K(X), V(X)$	由式 (1) 计算得到的 query、key、value。
$U(X)$	HSTU 中用于对注意力池化值进行门控的张量，使模型无需显式前馈层。
$A(X)$	输入 X 对应的注意力张量。
$Y(X)$	HSTU 层的输出。
$\text{Split}(\cdot)$	把大张量切分成若干块的操作。
$r_{ab}^{p,t}$	同时编码相对位置与相对时间信息的注意力偏置。
α	HSTU 中 Stochastic Length 算法控制稀疏度的参数。
R	HSTU 稀疏注意力算法上下文中的 GPU 寄存器大小。
m	排序阶段考虑的候选项数量。
b_m	M-FALCON 中的微批大小。

B 附录 B：生成式推荐器的背景与问题表述

许多读者可能更熟悉传统 DLRM：这一范式从 YouTube DNN 时代开始流行，并广泛应用于大型内容平台与电商平台。DLRM 在异构特征空间上工作，通常同时包含特征交互模块、顺序池化或目标感知成对注意力模块，以及多专家多任务模块。正文第 2 节和第 3 节已经从与 DLRM 对照的角度介绍了 GR；附录这里则进一步从传统序列推荐文献的视角重新展开。

B.1 学术界与工业界中的序列推荐背景

学术研究：传统序列推荐设定。 循环神经网络最早通过 GRU4Rec 被系统引入推荐领域。GRU4Rec 在两个数据集上使用 GRU，仅保留用户的正向行为作为序列输入，例如点击过的商品或观看时长达到阈值的视频。作者指出，若从现代工业两阶段推荐系统（召回 + 排序）的角度看，这类设定更接近于召回任务。

随后，Transformer 及其他序列转导架构的发展推动了推荐领域的进一步演化。SASRec 首次把 Transformer 用于自回归序列推荐；它把评论或评分的存在看作正反馈，把 Amazon Reviews、MovieLens 等经典数据集转成正样本物品序列，并以二元交叉熵训练“下一个正物品”的预测。之后的大量工作，包括 BERT4Rec、S3Rec 等，大都建立在与 GRU4Rec / SASRec 相似的设置上。

工业应用：作为 DLRM 的组成模块。 在工业界，顺序编码器与成对注意力模块因为能够增强用户表示，因此被广泛嵌入 DLRM 中使用。但这些系统一般只使用相对较短的序列长度，例如 BST 用 20，DIN 用 1000，TransAct 用 100，而本文的 GR / HSTU 则把长度推进到了 8192。

尽管序列长度较短，大多数 DLRM 仍然能够一定程度上捕获长期偏好，原因主要有两点：其一，现代 DLRM 常常配合预计算用户画像或外部向量库使用，本质上延长了可回溯窗口；其二，工业系统会叠加大量上下文特征、用户特征、物品特征，并结合 FM、DCN、MoE 等多种异构网络共同建模。

此外，工业主流工作大多把损失定义在“用户/请求-候选物品”这一对上。排序任务一般使用多任务二元交叉熵，召回任务则通常采用 two-tower 结构。近年的一些研究也尝试把“下一件推荐物品”表示成一个子 token 序列上的概率分布，这类设定有时也被称为 generative retrieval。

作者从问题表述角度认为，上述工业工作都应归入广义 DLRM 范式，因为其模型结构、使用特征与训练损失与学术界传统序列推荐依然存在明显差异。作者还特别指出：在本文之前，工业界并没有成功的大规模全序列化排序系统，尤其没有十亿 DAU 级别的成功实践。

B.2 GR 如何把排序与检索表述为序列转导任务

作者认为，传统序列推荐与 DLRM 在问题表述上都存在若干限制，而 GR 正是针对这些限制提出的。

第一，传统序列推荐通常只看“用户明确交互过的物品”，而忽略大量其他特征；工业推荐系统在这方面则恰恰相反，它们广泛依赖海量辅助特征来增强用户和内容表示。GR 的解决方案有两步：一是把其他类别型特征压缩后并入主时间序列；二是通过目标感知交叉注意力，在序列架构中隐式恢复数值型特征的作用。实验也验证了这一点：如果采用只看交互历史的“interaction-only”设定，检索 hit rate@100 会下降约 1.3%，排序 NE 也会显著变差。

第二，传统序列推荐通常在**目标无关**的设置下计算用户表示：对于目标物品 Φ_i ，编码器只看 $\Phi_0, \Phi_1, \dots, \Phi_{i-1}$ 。而工业排序系统普遍需要**目标感知**表达，即候选项信息必须参与用户表示的形成。DIN、BST、TWIN、TransAct 等工业方法都遵循这一思路。GR 通过“内容-动作交织”的输入形式，把目标项自然嵌入因果自回归框架，从而实现了目标感知注意力，同时又保留了纯序列建模的优点。

第三，传统序列推荐本质上是**判别式**的。它们通常建模 $p(\Phi_i | \Phi_0, a_0, \dots, \Phi_{i-1}, a_{i-1})$ ，即“给定当前状态，下一件推荐物品是谁”。而作者认为，完整推荐系统其实包含两个随机过程：其一是系统向用户展示内容 Φ_i ，其二是用户对该内容作出动作 a_i 。若要真正采用生成式方法，就应建模联合分布

$$p(\Phi_0, a_0, \Phi_1, a_1, \dots, \Phi_{n_c-1}, a_{n_c-1}).$$

GR 正是为此而设计的。它既可以做“下一动作 token 预测”，也可以做“下一内容 token 预测”，从而对输入数据分布本身建模。作者进一步推测，这一表述将来还有望直接通过 beam search 生成整段推荐列表，潜在上优于传统 listwise 推荐框架；不过这一方向仍留待未来工作深入展开。

C 附录 C：合成数据实验 (Synthetic Data)

正文第 3.1 节指出，标准 softmax 注意力由于其归一化方式，难以直接表达用户偏好强度，而这一点对推荐场景非常重要。为了更清楚地理解这种现象，作者构造了一个基于 Dirichlet Process 的合成数据集，用于模拟动态词表下的流式用户行为序列。Dirichlet Process 的“rich gets richer”性质，恰好与用户行为历史中“热门兴趣反复出现”的特点相吻合。

实验设置如下：首先，把 20,000 个 item id 随机划分到 100 个类别；然后生成 1,000,000 条长度为 128 的记录，其中前 90% 用于训练，后 10% 用于测试。为了模拟流式训练，开始时只有前 40% 的 item id 可见，其余 id 随数据流逐步开放。对每条记录，再随机挑选不超过 5 个类别，并基于一个先验分布 H_c 来依次采样序列中每个位置的类别。当 $n > 1$ 时，

以概率 $\alpha/(\alpha + n - 1)$ 从先验 H_c 抽取类别；以概率 $n_c/(\alpha + n - 1)$ 沿用历史中已出现过的类别，其中 n_c 表示该类别已出现次数；随后再在满足流式约束的前提下，从该类别下随机抽取具体 item。参数 α 从 (1.0, 500.0) 中均匀采样。

结果见原文表 2。由于该数据集没有时间戳，作者在 HSTU 中去除了相对时间偏置。实验表明，HSTU 相比标准 Transformer 在 Hit Rate@10 上提升超过 100%。同时，若把 HSTU 的逐点注意力替换成 softmax，命中率又会明显下降，这进一步验证了逐点聚合机制的重要性。

D 附录 D：传统序列推荐设置下的进一步评估

正文第 4.1.1 节主要把 HSTU 与强 Transformer 基线 SASRec 进行了比较。附录中，作者进一步纳入另外两类方法：其一是经典 RNN 序列推荐模型 GRU4Rec，用于帮助读者理解“在引入现代训练与工程技巧之后，自注意力模型与传统 RNN 的差异”；其二是最具代表性的自监督序列推荐模型 BERT4Rec，用于比较双向自监督（Cloze 目标）与单向因果自回归训练之间的差别。

表 12 显示，SASRec 在 sampled softmax 训练条件下仍然是传统序列推荐中非常强的基线，这与近期研究结论一致；但 HSTU 依然显著超过已评估的 Transformer、RNN 和双向自监督 Transformer。作者在此保持了负样本数量一致，以确保比较公平。

E 附录 E：传统 DLRM 基线说明

正文第 4 节使用的 DLRM 基线，并非简单论文复现，而是大型互联网平台上经过多年迭代、接近真实生产配置的强基线。作者在此给出高层描述。

排序基线。 排序模型大约使用一千个 dense 特征和五十个 sparse 特征，并融合了多种成熟建模技术，例如 Mixture of Experts、Deep & Cross Network 变体、各种顺序推荐模块（包括工业界常见的目标感知成对注意力），以及交互层上的残差连接等。在 scaling law 低 FLOPs 区间实验里，作者还对一些高成本模块做了简化或替换，以控制整体 FLOPs。

虽然由于保密要求无法公开全部细节，但作者认为，这一基线已经接近现有公开方法全面融合后的最强 DLRM。为帮助读者理解，作者还另外构造了一个只使用主要公开模块（DIN + DCN + MMoE）的 DLRM 变体，并在图 9 中给出了该结构示意。这个“公开模块版”基线在主任务上明显弱于真实生产 DLRM。

检索基线。 检索模型采用标准 two-tower 结构，结合批内与批外负采样。输入特征既包含高基数 sparse 特征（如 item id、user id），也包含低基数 sparse 特征（如语言、话题、兴趣实体），随后通过带残差连接的前馈层压缩成用户塔与物品塔表示。

特征与序列长度。 作者特别强调，两类 DLRM 基线中使用的特征集合，包括交互历史序列，都是本文所有 GR 模型输入特征的真超集。这意味着 GR 并非“凭借更多特征获胜”，相反，它是在更简洁的特征体系上取得更强结果。

F 附录 F: Stochastic Length

F.1 子序列选择 (Subsequence Selection)

在正文公式 (4) 中, Stochastic Length 需要从完整用户历史中选取一个长度为 L 的子序列。作者发现，**如何选取**这个子序列，会直接影响模型质量。为此，作者定义特征 $f_i = t_n - t_i$ ，表示用户与 item x_i 交互距当前时刻已经过去的时间，并比较了三种策略：

1. **贪心选择**：从集合 S 中挑选 f_i 最小的 L 个条目，即最近的若干交互；
2. **随机选择**：从集合 S 中随机抽取 L 个条目；
3. **特征加权选择**：按与时间特征相关的加权分布进行抽样。

离线实验表明，特征加权的選擇方式能带来最佳模型质量，原文表 13 给出了对应结果。

F.2 Stochastic Length 对序列稀疏性的影响

正文表 3 给出了在 30 天用户历史配置下，Stochastic Length 对序列稀疏性的影响。这里，作者把“序列稀疏性”定义为 $1 - (\text{平均序列长度} / \text{最大序列长度})$ 。为了更准确刻画稀疏注意力的真实计算负担，作者还定义了 s^2 指标，可理解为注意力矩阵稀疏性的补充度量。附录中的表 14 与表 15 则分别给出了 60 天和 90 天用户历史下的对应统计。

F.3 与序列长度外推技术的比较

作者还将 Stochastic Length 与语言建模中常见的序列长度外推方法做了对比。很多已有方法都是通过修改 RoPE 位置编码来实现更长上下文的泛化，因此作者训练了一个不带相对偏置、而带旋转位置编码的 HSTU-RoPE 作为对照。比较方法包括：

1. 对 HSTU-RoPE：零样本外推（直接应用 NTK-aware RoPE 后测试）与微调外推（应用 NTK-by-parts 后再微调 1000 步）；
2. 对标准 HSTU：零样本外推（直接截断/钳制相对位置偏置）与微调外推（钳制后再微调 1000 步）。

原文表 16 报告了这些方法与“完整长度训练”相比的 NE 差值。作者认为，在推荐这类高基数 ID 场景中，零样本和微调式长度外推都不理想。经验上看，Stochastic Length 明

显优于这些方法。作者推测，这可能与推荐中的超大词表有关：当训练长度较短时，旧 ID 的表示学得不充分，导致模型难以真正利用更长历史中的信息。

G 附录 G：稀疏分组 GEMM 与融合相对注意力偏置

本节补充介绍正文第 3.2 节中高效 HSTU 注意力核的实现细节。作者的方法建立在 Memory-efficient Attention 与 FlashAttention 基础之上，本质上是一种把输入划分为块、并避免在反向传播中显式物化巨大 $h \times N \times N$ 中间注意力张量的内存高效自注意力机制。通过利用输入序列的稀疏性，注意力计算可以被重写为一组形状不同、前后相接的 GEMM 运算，然后作者再通过定制 GPU kernel 对其加速。

作者还指出，相对注意力偏置本身的构造也会带来额外内存访问开销。为解决这一问题，他们把偏置构造与 grouped GEMM 融合进单个 GPU kernel，并在反向传播中利用 GPU 的高速 shared memory 进行梯度累积。虽然这种实现要求在反向过程中重新计算注意力与相对偏置，但相较于标准 Transformer 实现，它依然明显更快、占用更少内存。

H 附录 H：M-FALCON 详解

本节详细介绍正文第 3.4 节中的 M-FALCON 算法，并在原文算法 1 中给出了伪代码。作者认为，M-FALCON 主要包含三项关键思想。

1. 因果自回归设置下也可以做批量推理

GR 的排序任务是目标感知的。通常直觉认为，在目标感知设定下，每个候选项都必须单独做一次前向传播，因此当候选数为 m 、序列长度为 n 时，成本是 $O(mn^2d)$ 。作者指出，这并不是最优实现：即便对普通 Transformer，只要修改自注意力中的 mask，也可以把多候选推理批量化，并把复杂度降低到 $O((n+m)^2d)$ ，在常见推荐配置下近似视作 $O(n^2d)$ 。

其基本做法是：保留历史序列部分的因果掩码，同时对同一个微批中的候选位置之间强制设置“不可互相注意”。这样，最后每个候选位置的输出都只依赖公共历史，而不依赖其它候选，从而一次前向传播就等价于做了多个“单候选”前向传播。

2. 微批处理把批量推理扩展到大候选集合

排序阶段往往需要同时评估上万候选项。作者因此把总共 m 个候选分成 $\lceil m/b_m \rceil$ 个微批，每个微批大小为 b_m ，并令 $O(b_m) = O(n)$ 。这样一来，整体时间复杂度在实际推荐场景下仍可近似维持在 $O(n^2d)$ 。

3. 编码器级缓存使跨微批与跨请求共享成为可能

最后，作者将 KV cache 同时应用于“微批之间”与“不同请求之间”。例如，对本文中的 HSTU 而言，历史序列对应的 $K(X)$ 与 $V(X)$ 可以在多个微批间复用；当执行带缓存的前向传播时，只需为最后 b_m 个候选计算新的 $U(X)$ 、 $Q(X)$ 、 $K(X)$ 、 $V(X)$ ，而用户历史部分的 $K(X)$ 、 $V(X)$ 可以直接复用。输出端的逐点变换同理，也只需对候选位置重新计算。这会把一次缓存前向的复杂度降为

$$O(b_m d^2 + b_m n d),$$

相较于未缓存的

$$O((n + b_m)d^2 + (n + b_m)^2 d),$$

即便在 $b_m = n$ 的情况下，也能带来 $2\times-4\times$ 的额外收益。

算法 1: M-FALCON 中文化步骤

为便于阅读，这里用文字重述原文算法 1 的流程。

1. 输入：合并后的 token 序列 $x_0, x_1, \dots, x_{n-1}$ ，以及 m 个排序候选 $\Phi'_0, \dots, \Phi'_{m-1}$ ；一个在因果自回归设置下训练好的、具有 b 层 h 个头的自注意力模型 f ；微批大小 b_m 。
2. 先计算微批数目 $\text{numMicrobatches} = (m + b_m - 1) // b_m$ 。
3. 构造大小为 $n + b_m$ 的下三角注意力矩阵。
4. 对所有满足 $i, j \geq n$ 且 $i \neq j$ 的位置，把注意力掩码设为 $-\infty$ ，从而阻止同一微批中的不同候选之间互相注意。
5. 将历史序列与第一个微批候选拼接后送入模型，得到第一批预测，同时把 KV 缓存保存下来。
6. 之后对其余微批重复上述过程：每次把新的候选微批与历史序列拼接，调用带缓存的模型前向，得到该微批预测并追加到总输出。
7. 当所有微批处理结束后，返回全部候选的预测结果。

作者强调，M-FALCON 不仅适用于 HSTU 与 GR，也可以更广泛地作为“目标感知、因果自回归、自注意力架构”的通用推理加速算法。

H.1 吞吐实验：带 M-FALCON 的 GR 与 DLRM

最后，作者在相同硬件环境下，比较了 GR 与 DLRM 在大规模工业设置中的候选打分吞吐（QPS）。结果见原文图 12 与图 13。实验表明，随着排序候选数 m 增加，GR 的吞

吐先呈现明显的次线性扩张，这正是批量化推理能够摊销计算成本的直接体现。进一步地，由于注意力复杂度随 $(n + b_m)^2$ 增长，采用多个微批本身就能提高吞吐，而缓存又能进一步消除重复的线性层与注意力计算。两者结合时，相比 $b_m = m = 1024$ 的单微批基线，作者观察到最高 $1.99\times$ 的额外加速。

综合来看，借助高效的 HSTU 编码器设计以及 M-FALCON，基于 HSTU 的生成式推荐模型虽然在 FLOPs 上比 DLRM 高出 285 倍，但在真实生产级配置中，吞吐仍可最高领先 $2.99\times$ 。这从另一个侧面说明，本文的贡献并不只是“模型更大”，而是在 **问题表述、体系结构与系统实现** 三个层面同时协同优化。

参考文献说明

原论文参考文献数量较多，且主要用于文献索引与追溯。为保证排版紧凑，本中文翻译整理版未逐条翻译参考文献条目；如需精确引用，请以原论文参考文献页为准。